

Release management

Week 5, Lecture 2 · B2C to \$10k MRR: A 10-Week Growth Manual

Autumn 2026

What we will cover

- CI/CD for one person: auto-deploy on green main
- Feature flags: PostHog, Statsig, or a hand-rolled config table
- Phased rollouts: 1% to 10% to 100% with rollback criteria at each gate
- The kill-switch pattern: decision to reverted prod in 5 minutes
- TestFlight and Play internal testing as your safety net

Part 1: CI/CD for one person

The goal: deploy on green main

- Every push to main that passes tests deploys to production automatically
- No manual deploy steps, no "I need to remember to push"
- Failure on any gate (lint, type check, test) blocks the deploy and notifies you
- Shape Up principle (Singer, 2019): "done means deployed." Integration is the definition of shipped.

A minimal GitHub Actions pipeline

```
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run build    # fails the deploy if build breaks
      - run: npm test
      - uses: vercel/actions/deploy@v1
```

Three gates: install, build, test. If any fails, no deploy.

Why this matters at \$0 MRR vs. \$10k MRR

- At \$0 MRR: a broken deploy wastes your day, not a customer's subscription
- At \$10k MRR: a broken deploy is a support ticket, a refund, a churn event
- Building the habit when the stakes are low means the pipeline is already there when stakes are high
- PostHog (2024): feature flags + CI/CD is how teams maintain deploy velocity as user base grows

Part 2: feature flags

What a feature flag does

- A feature flag is a conditional in your code controlled by an external config
- `if (flag.isEnabled('new-pricing-page', user)) { ... }`
- The config lives in PostHog, Statsig, or your own database table
- You change the config without redeploying code

Three uses for feature flags this week

- **A/B testing:** randomly assign 50% of users to a variant
- **Phased rollout:** expose 1% then 10% then 100% of users over days
- **Kill switch:** set flag to 0% to instantly revert a bad release

All three are the same underlying mechanism with different percentages.

Hand-rolling a feature flag (if you want to avoid a third-party dependency)

```
-- A flags table in your database
CREATE TABLE feature_flags (
  key TEXT PRIMARY KEY,
  rollout_pct INTEGER DEFAULT 0,
  enabled BOOLEAN DEFAULT FALSE
);
```

Check: `hash(user_id + flag_key) % 100 < rollout_pct`. No external SDK needed.

Part 3: phased rollouts

The 1% to 10% to 100% ladder

- **1%:** internal users, beta testers, or a random 1% slice. Watch error rates.
- **10%:** broader exposure. Check activation rate, session replay, support tickets.
- **100%:** only after the 10% gate looks clean for at least 24-48 hours.

PostHog recommends starting at 5-10% and monitoring before expanding (PostHog, 2024).

What to watch at each gate

At 1% and 10% gates, check these before advancing:

- Error rate: same as or lower than baseline?
- Activation rate: same as or higher?
- Session replays: any obvious confusion or drop-offs?
- Support ticket volume: any new category appearing?

If anything looks wrong: set the flag to 0% before you investigate.

Rollback criteria: write them before you ship

- Define before the 1% rollout: "I will revert if error rate rises above X% or activation drops below Y%"
- Written down means you decide when you are calm, not when you are panicking
- The criteria also tell you when you are safe to advance to the next gate
- Statsig (2026): pre-defined rollback criteria are what separate a phased rollout from just releasing to everyone slowly

Part 4: kill switches

What a kill switch is

- A kill switch is a feature flag set to 0%: instant rollback without a new deploy
- Useful for: crash loops, bad UI states, incorrect pricing, broken payment flows
- Time from decision to rollback: under 60 seconds if the flag is already in place
- Statsig (2026): "the difference between a bad hour and a bad week is often just having a kill switch ready"

The rollback runbook

A 1-page document that answers four questions:

1. Who makes the rollback decision? (You. Define the threshold in advance.)
2. What is the exact command or dashboard action?
3. How do you confirm the rollback succeeded? (Check one metric in 5 minutes.)
4. What do you post to users if they were affected?

Write this before shipping the feature, not after the incident.

Practicing the rollback drill

- Set up a test flag in staging or prod pointing to a harmless variant
- Time yourself going from "I decide to revert" to "I confirm it is reverted"
- 5 minutes is the target; the first drill usually takes 15-20 minutes
- The drill exposes gaps in your runbook before you need it under pressure

Part 5: mobile safety nets

TestFlight and Play internal testing

- TestFlight (iOS): distribute to up to 10,000 testers before App Store submission
- Play internal testing (Android): release to a closed group before public track
- Both let you catch crashes, UI regressions, and flow breaks before 100% of users see them
- Use these as your mobile equivalent of the 1% gate on web

When to use each track

- **Internal testing:** your own devices + QA. Every build before any external user sees it.
- **TestFlight external / Play closed testing:** beta users who opted in. Treat like your 10% gate.
- **App Store / Play production:** only after both gates are clean.
- Keep the tracks in sync with your feature flags: flag off for most users, flag on for testers.

Integrates one slice at a time, clear definition of done. Done means deployed.

<footer>Ryan Singer, Shape Up (Basecamp, 2019)</footer>

Take-aways

1. Auto-deploy on green main removes the "I forgot to deploy" failure mode and forces you to keep tests passing.
2. Feature flags are not an advanced technique: a single database column and a hash function is enough to get the benefits.
3. Write rollback criteria before the 1% gate, not after something breaks at 100%.
4. A kill switch is a feature flag at 0%. The rollback runbook is the document you write when you are calm, for the moment you are not.
5. TestFlight and Play internal testing are your mobile 1% gate. Use them for every build.

What's next

- **Reading:** Week 5 reading (also covers the Friday mid-course check)
- **Section (this week):** Design one A/B test with sample-size math
- **Friday:** 60-minute mid-course pivot/persevere check
- **Week 6:** Channel selection: pick one channel and commit for weeks 6-10